CKP7500 - Sistemas Distribuídos e Redes De Comunicação
SMD - Sistemas Distribuídos
Prof. Windson Viana de Carvalho
Prof. Fernando Antonio Mota Trinta
2026

Invocação Remota e Comunicação Indireta

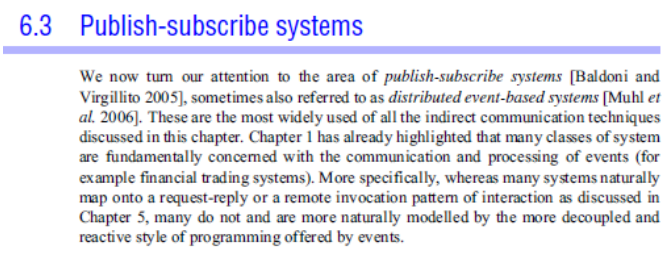
Objetivos do Roteiro

O objetivo principal do roteiro é explicar a atividade comparativa entre os paradigmas de comunicação. Além disso, o roteiro contém um material de estudo sobre o modelo Pub-Sub e MQTT.

Tempo de estudo : 2h Tempo de Atividade: 6h-10h

1 - O Modelo Publish-Subscribe

Um sistema do tipo Publish-Subscribe é um sistema em que parte dos comunicantes publicam eventos para um serviço de eventos e os assinantes manifestam interesse em eventos específicos por meio de assinaturas que podem ser padrões arbitrários sobre os eventos estruturados (e.g., ouvir eventos do tipo X, ouvir eventos do tipo Y se parâmetro Z for maior que W). Mais especificamente, um assinante pode expressar interesse em todos os eventos relacionados ao livro de Sistemas Distribuídos da disciplina, como disponibilidade de uma nova edição ou atualização do site relacionado ao livro.



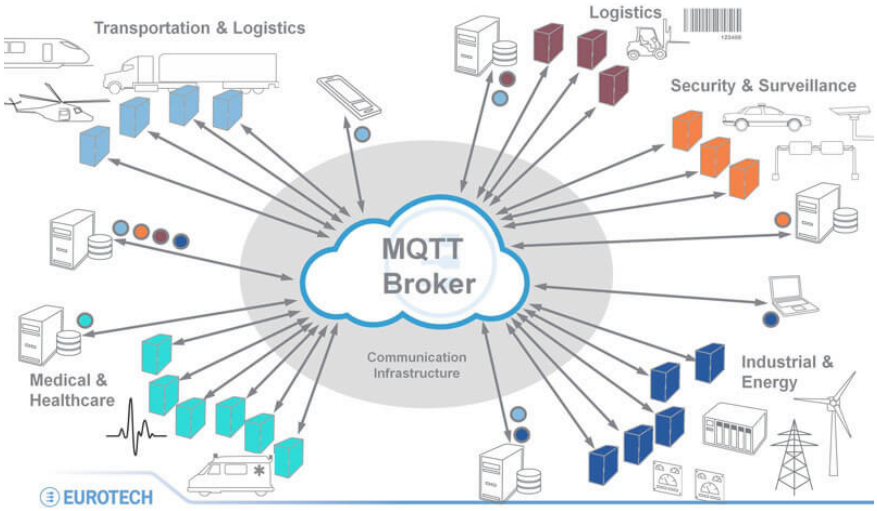
6.3 Publish-subscribe systems

We now turn our attention to the area of *publish-subscribe systems* [Baldoni and Virgillito 2005], sometimes also referred to as *distributed event-based systems* [Muhl *et al.* 2006]. These are the most widely used of all the indirect communication techniques discussed in this chapter. Chapter 1 has already highlighted that many classes of system are fundamentally concerned with the communication and processing of events (for example financial trading systems). More specifically, whereas many systems naturally map onto a request-reply or a remote invocation pattern of interaction as discussed in Chapter 5, many do not and are more naturally modelled by the more decoupled and reactive style of programming offered by events.

A tarefa do sistema publish-subscribe é de confrontar as inscrições com os eventos publicados e garantir a entrega correta de notificações de eventos que ocorreram àqueles interessados em sua observação. Um determinado evento pode ser entregue a muitos assinantes, e, portanto, o modelo Publish-Subscribe é fundamentalmente um paradigma de comunicação indireta do tipo um-para-muitos conforme pode ser lido no **Capítulo 6** do livro principal da nossa bibliografia (i.e., Couloris).

Sistemas deste tipo são usados em uma grande variedade de domínios de aplicação, particularmente aqueles relacionados à grande escala de divulgação de eventos e ou em arquiteturas de entrega de mensagens por canais como no Twitter/X.

A figura ao lado representa um exemplo de implementação desse estilo arquitetural na tecnologia MQTT que se baseia em eventos (no caso, chamados de tópicos) para a comunicação.



Note que parte dos elementos comunicantes produz e envia dados para o broker e esse se encarregará de entregar as mensagens corretamente para os participantes que se subscreveram para um determinado tópico sem que haja dependência direta ou comunicação direta entre os comunicantes.

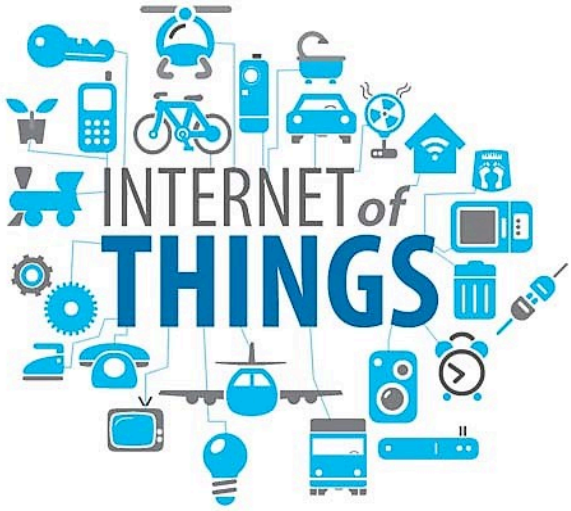
.

Exemplos de cenário em que esse tipo de comunicação pode ser usada incluem:

I. Sistemas de informação financeira;
II. Sistemas Web com feeds de dados em tempo real (incluindo feeds RSS);
III. Apoio ao trabalho cooperativo, onde um número de participantes precisa ser informado de eventos de interesse compartilhado;
IV. Suporte para o gerenciamento de eventos em Sistemas Ubíquos e de Internet das Coisas que necessitem desacoplamento entre sensores e atuadores
V. Um amplo conjunto de aplicações de monitoramento, incluindo monitoramento de rede na Internet.

1.1 Exemplo de cenário - Internet das Coisas

Internet das Coisas, do inglês Internet of Things (IoT), é um paradigma tecnológico que emerge no cenário já consolidado das redes de comunicações sem fio. A ideia principal desse conceito é a constante e invisível presença, no cotidiano das pessoas, de uma enorme variedade de dispositivos computacionais.



Alguns desses dispositivos possuem poder computacional, juntamente com a capacidade de comunicação e uso direto ou indireto dos protocolos da pilha TCP/IP. Sensores, atuadores e muitas vezes uma junção deles são chamados de Objetos Inteligentes (OI) ou Smart Objects. Esses objetos são unicamente endereçáveis, capazes de interagir, trocar dados entre si e ainda cooperar com seus vizinhos para realizarem tarefas em comum. São exemplos de OIs: lâmpadas inteligentes, assistentes de voz como a Alexa da Amazon e SmartTVs.

Ao longo dos últimos anos, o volume mundial de OI cresceu rapidamente. De acordo com a Cisco, a marca de 50 bilhões de OI conectados foi atingida em 2020. Assim, o número de dispositivos conectados à rede pode já ter passado de 13,6 por habitante da terra. Grande parte desses OI já são compatíveis com arquiteturas e protocolos Web (HTTP, REST, MQTT entre outros).

Outra forma de ver a Internet das coisas é proposta por Gubbi et al.[1] que definem IoT como sendo a interconexão de sensores e atuadores provendo a habilidade de compartilhar informação ao longo de plataformas por meio de um framework unificado, desenvolvendo um cenário operacional comum para permitir aplicações inovadoras . Isso é alcançado por um sensoriamento ubíquo, análise dos dados e representação da informação, tendo a Computação em Nuvem como o framework unificador. Já objetos inteligentes podem ser vistos como dispositivos que possuem identidade e

[1] GUBBI, J.; BUYYA, R.; MARUSIC, S.; PALANISWAMI, M. Internet of things (iot): A vision, architectural elements, and future directions.Future Generation Computer Systems, v. 29, n. 7, p. 1645 – 1660, 2013. ISSN 0167-739X. Disponível em: <http://www.sciencedirect.com/science/article/pii/S0167739X13000241>.

personalidade virtual e que operam em ambientes inteligentes usando interfaces para conectar e se comunicar dentro de um contexto social, local e pessoal.



O NIC.br tem um vídeo bem interessante explicando o conceito de IoT. Assista em A Internet das coisas, explicada pelo NIC.br (7 minutos).

Outro vídeo interessante é apresentado pela Cisco em Fundamentals of the Internet of Things (IoT) (13 minutos)

Outro aspecto importante é a utilização de padrões e protocolos bem especificados para a comunicação com, e entre, esses OIs, buscando assim atingir a máxima interoperabilidade possível entre os mais diversos dispositivos. Dentre essas especificações, destacam-se as que se adéquam às características comuns dos OIs, como baixo poder computacional, volatilidade (entrada e saída do ambiente) e limitações energéticas. Limitações essas que fazem com que protocolos amplamente adotados na WEB tradicional, como o HTTP, percam espaço para protocolos mais adequados aos cenários de IoT, como o CoAP e o MQTT. Sendo o último objeto de estudo dessa aula.



Uma sugestão de leitura importante com desafios que a IoT traz para o desenvolvimento de software pode ser lido em Taivaissari et al. em A Roadmap to the Programmable World: Software Challenges in the IoT Era

1.2 Desacoplamentos Espacial e Temporal

O modelo Publish-Subscribe é um exemplo de implementação de comunicação indireta e pode garantir desacoplamento temporal e espacial entre os comunicantes se corretamente implementado (para a revisão desses conceitos veja a tabela a seguir). Vale ressaltar que a implementação não é trivial caso não se utilize a versão centralizada do broker. Neste tipo de design arquitetural, o broker centralizado representaria um ponto crítico de falha e um gargalo de comunicação, além de ser

fortemente acoplado às demais partes do sistema. Entretanto, a implementação distribuída é mais complexa e incorre em maiores dificuldades na troca de mensagens e garantia dos estados uma vez que os elementos componentes do sistema podem ser voláteis em um determinado ambiente.



Acoplamento e Desacoplamento em Sistemas Distribuídos

Fonte: https://pt.slideshare.net/victorhazin/aula-de-sistemas-distribudos-comunicao-indireta/6

1.3 MQTT

Como explicado anteriormente, para os dispositivos de Internet das Coisas (IoT), a conexão com a Internet é um requisito seja ela diretamente usando TCP/IP ou por meio de uso de proxies ou redes subjacentes como a 6LowPan. Desenvolvido com base na pilha TCP/IP, o MQTT (Message Queue Telemetry Transport) tornou-se um padrão de fato para comunicações de IoT.

O MQTT foi inventado e desenvolvido inicialmente pela IBM no final dos anos 90[2]. Sua aplicação original era vincular sensores em pipelines de petróleo a satélites. Como seu nome sugere, ele é um protocolo de mensagem com suporte para a comunicação assíncrona entre as partes. Um protocolo de sistema de mensagens assíncrono desacopla o emissor e o receptor da mensagem tanto no espaço quanto no tempo e, portanto, é escalável em ambientes de rede que não são confiáveis. Apesar de seu nome, ele não tem nada a ver com filas de mensagens, na verdade, ele usa um modelo de publicação e assinatura. No final de 2014, ele se tornou oficialmente um padrão aberto OASIS, com suporte nas linguagens de programação populares,

[2] https://www.ibm.com/developerworks/br/library/iot-mqtt-why-good-for-iot/index.html

usando diversas implementações de software livre. Na tabela a seguir, sugerimos algumas leituras sobre o MQTT e o Paho, projeto Eclipse para programação usando MQTT.

Tema	Links
[PT] Conhecimentos básicos de MQTT e introdução ao Paho[	Conhecendo o MQTT – IBM Developer Configurando o broker de publicação/assinatura de MQTT integrado
[EN] Introdução ao Paho - Pelo eclipse	Eclipse Paho | The Eclipse Foundation
[EN] Documentação MQTT	MQTT Version 5.0
[PT] Slides da aula feita pelos professores da disciplina	2026-Aula04_Revisao_Paradigmas_de...

2 - Vídeos sobre MQTT

No complemento das aulas sobre Paradigmas de Comunicação, o professor Windson gravou um vídeo explicando um exemplo de funcionamento do MQTT usando a biblioteca Paho e um broker público. Além desse vídeo da aula, vocês podem conferir outros vídeos sobre a implementação do MQTT e ferramentas associadas (acessem principalmente quando forem programar).

Tema	Links	Duração
Vídeo do MQTT (Prof. Windson)	Vídeo	21:45
[PT] Protocolo MQTT (vídeo de alunos)	Protocolo MQTT	11:44
[PT] Instalando e Testando o Mosquitto (MQTT)	Instalando e Testando ...	16:07
[EN] IoT using Eclipse Paho for Java, Mosquitto MQTT	IoT using Eclipse Paho f...	28:04

Atividade AT03 - Programação da Calculadora Distribuída
Data de Entrega: 30 de Outubro
Grupos de até 4 pessoas

Preâmbulo

A ideia dessa atividade é permitir um melhor entendimento dos códigos de comunicação exibidos pelos professores nas aulas de Paradigmas de comunicação como os códigos de Socket, RMI, JSON-RPC, HTTP/REST e MQTT. Ela é composta de 5 projetos distintos. Quatro deles já existem e estão atualmente invocando ou implementando apenas a operação de soma de uma calculadora.

Avaliando meus conhecimentos - 1 ponto (Individual)

Mas antes de começar a atividade de programação, você deve responder a esse pequeno questionário que avalia os conhecimentos explicados nas aulas e neste roteiro: https://forms.gle/3oErmX2PY7ifY3348

Atividade de Programação + Relatório - 9 pontos (até 4 pessoas)

A tarefa consiste em implementar uma calculadora remota com outras operações básicas além da soma (subtração, divisão e multiplicação) e implementar o cálculo de expressão aritméticas usando duas abordagens (1- a que interpreta no cliente e faz as múltiplas chamadas ao servidor, 2- a que envia a expressão pro servidor calcular). Além disso existe um projeto MQTT a ser feito **(detalhes a seguir)**.

Requisitos Calculadoras

1- Os clientes das calculadoras devem ter uma interface gráfica mínima (textual ou visual)
2- Devem suportar operações individuais (e.g., 10 + 15) como expressões matemáticas com parênteses (e.g., (10 + 15) * 4). As expressões devem ter duas implementações (uma em que são **decompostas nos clientes** que invocam as operações básicas no servidor e outra onde é calculada inteiramente no servidor).
3- Você pode utilizar os projetos dos professores, mas deve comentar cada linha do código das classes de comunicação indicando o que a linha de comando faz.
4- Podem escolher outras linguagens de programação
5- O cliente HTTP-REST deve implementar **uma política de RETRY** (e.g., Wait Ts and Retry)

Tipos e Plataformas

1- Sockets em Java ou em outra plataforma - 0,5 pontos

Comentar e evoluir os projetos JavaClientSocket_Calculadora e JavaServerSocket_Calculadora acrescentando as outras operações da calculadora e a interface de suporte a expressão matemática. Podem ser refeitos em outras linguagens de programação.

- Sockets: ClientSocket_Calculadora e ServerSocket_Calculadora

2- RMI em Java ou em outra plataforma (Kotlin RPC ou Pyro) - 0,5 pontos

Comentar e evoluir o cliente e o servidor, a interface e o objeto remoto do projeto ExemploRMI para tratar as outras operações da calculadora e a execução das expressões matemáticas

- RMI: https://github.com/windcarvalho/codigosSD/tree/master/ExemploRMI

3- JSON-RPC - 1 ponto

Comentar e evoluir o cliente e o servidor do projeto escolhido adicionando as requisições e tratamento das respostas das outras operações da calculadora.

- JSON-RPC: https://github.com/windcarvalho/rpcKTOR

3- HTTP-REST simples - 2 pontos

(0.5 pontos) Comentar e evoluir o cliente HTTP do projeto escolhido adicionando a requisição e tratamento das respostas das outras operações da calculadora.
(0.5 pontos) Comentar e evoluir a página PHP (ou em outra linguagem) para calcular as outras operações por meio de POST.
(1 ponto) **Implementar uma política de Retry**

- HTTPS: HTTPSCalculadoraPython ou ClientHTTPSCalculadora
- A Calculadora Servidora: CalculadoraPHP

4- Projeto MQTT - 3 pontos

Detalhes na Próxima página

Relatório Comparativo - 2 pontos

A tarefa consiste em criar um documento curto de até 8 páginas indicando os repositórios criados dos projetos, um desenho/diagrama comparativo das arquiteturas implementadas, as capturas dos tráfegos no Wireshark e uma análise comparativa dos modelos de comunicação (Socket, Requisição-resposta com HTTP, JSON-RPC, RMI e MQTT), dando ênfase às suas diferenças, vantagens e desvantagens em termos de

interoperabilidade, desacoplamento, desempenho e cenários mais adequados. Um PDF do documento deverá ser enviado pelo SIGAA.

Projeto MQTT - 3 pontos

Imagine um cenário em que um ambiente industrial (e.g., uma caldeira) é monitorado por um conjunto de sensores de temperatura que envia dados a cada 60 segundos para um broker MQTT. Ligado ao broker existem dois outros sistemas: o serviço **Compute Average Temperature (CAT)**, que calcula a média das temperaturas monitoradas e o serviço **Alarms** que dispara mensagens de vários tipos de alarmes (fogo, temperatura alta, vazamento de água) para operadores humanos do sistema.



O serviço **CAT** funciona como uma calculadora remota medindo a média dos valores de temperatura enviados nos últimos 120 segundos. Ele deve ser criado como um serviço que escuta - está inscrito para ouvir - tópicos de publicação de temperatura e executa a operação da média a partir do payload enviado no tópico. Esse serviço pode publicar duas mensagens: *aumento de temperatura repentina* se as duas últimas médias diferem em 5 graus e *temperatura alta* se a média indicar valor acima de 200 graus.

O serviço **Alarms** basicamente monitora esses dois eventos (i.e., as duas mensagens acima) e publica um alarme na interface gráfica do sistema de monitoramento quando eles ocorrem.
//
A **Atividade MQTT** é portanto implementar sensores simulados (o número de sensores pode aumentar ou diminuir) e os serviços **CAT** e **Alarms (1,5 pontos).** Este último pode

ser apenas um sistema que imprime mensagens no console ou pode ter uma interface gráfica **(0,5 pontos adicionais)**.

SUGESTÃO: Usar o MQTT Paho com o Mosquito para a implementação e criar 3 projetos distintos (um para o sensor e os outros dois para o serviços). Entretanto, vocês podem escolher outras plataformas e linguagem desde que usem um MQTT “raiz”.

Para a entrega coloque no Github do projeto um pequeno vídeo mostrando a execução, em sua máquina, dos sensores simulados enviando dados, do serviço CAT computando as médias e do serviço Alarms disparando pelo menos um dos alarmes (**1 ponto**)

ATENÇÃO ALUNOS DA PÓS: No caso de vocês, ao menos um sensor deve ser real (e.g., arduino ou um app de smartphone)